Muhammad Hayyan – 23i2041

Abdul Ahad – 23i2014

Muhammad Faizan – 23i2074

Security Documentation

Agentic AI-Driven Secure API Testing & Analysis Platform

Contents

Agentic AI-Driven Secure API Testing & Analysis Platform 1

1. Authentication Security 4
   - 1.1 Password Hashing with bcrypt (Cost Factor 12) 4
   - 1.2 Timing-Safe Password Comparison 4
   - 1.3 Password Hash Never Returned in API Responses 4
   - 1.4 Strong Password Policy Enforced at Schema Level 4
   - 1.5 Account Lockout After Repeated Failures 5
   - 1.6 JWT Secret Enforcement 5
   - 1.7 Two-Factor Authentication (TOTP) 5
   - 1.8 Secure 2FA Challenge Binding 5
2. Authorization & Access Control 5
   - 2.1 Session Verification Middleware (protect) 5
   - 2.2 Role-Based Access Control (authorize) 6
   - 2.3 Server-Side Role Promotion 6
   - 2.4 Object-Level Authorization (IDOR/BOLA Prevention) 6
   - 2.5 Notification Authorization (BOLA) 6
   - 2.6 Mass Assignment Protection 6
3. Rate Limiting & Brute-Force Protection 7
   - 3.1 Strict Rate Limit on Authentication Endpoints 7
   - 3.2 General API Rate Limit 7
   - 3.3 Progressive Request Slowdown 7
   - 3.4 Notification Polling Backoff 7
4. Input Validation & Injection Prevention 7
   - 4.1 Global Input Sanitizer (NoSQL + XSS) 7
   - 4.2 Why express-mongo-sanitize Was Replaced 8
   - 4.3 Path Traversal / Zip Slip Prevention 8
   - 4.4 Proper HTML Sanitization for Newsletter (Parser-Based) 8
5. Data Protection & Encryption 8
   - 5.1 AES-256-CBC Encryption for Source Code 8
   - 5.2 Unique IV Per Encryption 8
   - 5.3 Memory-Hard Key Derivation (scrypt) 9
   - 5.4 Mandatory Vault Credentials 9
   - 5.5 Sensitive Fields Not Returned by Default 9
6. Security Headers 9
   - 6.1 Helmet (Backend API) 9
   - 6.2 Frontend & Admin Security Headers 9
7. Session & Cookie Security 10
   - 7.1 HttpOnly Cookies 10

7.2 CORS Restricted to Known Origins 10
7.3 CSRF Protection via Ory Flow Tokens 10
8. API Security & Error Handling 10
8.1 Generic Error Messages (Anti-Enumeration) 10
8.2 No Stack Traces in API Responses 10
9. Frontend-Side Security 11
9.1 Avatar URL Validation 11
9.2 Minimal Sensitive Data in localStorage 11
9.3 Admin HTML Preview Sanitization 11
9.4 Download Filename Sanitization 11
9.5 ProtectedRoute Only Redirects on Real Auth Failures 11
10. Cloud & Environment Security 12
10.1 No Secrets in Source Code 12
10.2 NEXT_PUBLIC Prefix for Client-Exposed Variables 12
10.3 ORY_SDK_URL Required in Production 12
Summary Table 12

1. Authentication Security

1.1 Password Hashing with bcrypt (Cost Factor 12)

Passwords are never stored as plaintext. Every time a user sets or changes their password, the `pre-save` hook on the User model automatically hashes it using bcrypt with a cost factor of 12. Cost factor 12 means the hash takes roughly 250ms to compute, making brute-force attacks extremely slow.
Where: `BACKEND/src/models/User.js`

```
userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;
  this.password = await bcrypt.hash(this.password, 12);
});
```

1.2 Timing-Safe Password Comparison

When checking a login password against the stored hash, we use `bcrypt.compare()` rather than a regular `===` comparison. bcrypt.compare takes the same amount of time whether the password is wrong on character 1 or character 50, which prevents timing side-channel attacks.

Where: `BACKEND/src/models/User.js` — line 186

```
return await bcrypt.compare(candidatePassword,
userPassword);
```

1.3 Password Hash Never Returned in API Responses

The password field is defined with `select: false` in the schema. This means that unless code explicitly asks for the password field (which only the login and password-change functions do), it is automatically excluded from every database query result. Users can never accidentally leak their own hash through the API. Where: `BACKEND/src/models/User.js` — line 84

```
password: { type: String, select: false,
... }
```

1.4 Strong Password Policy Enforced at Schema Level

The password schema validator checks the new password before the bcrypt hook even runs. It requires at least 8 characters, one uppercase letter, one number, and one special character. This runs on every save where the password was modified.

Where: `BACKEND/src/models/User.js` — lines 85–95

1.5 Account Lockout After Repeated Failures

After 5 consecutive failed login attempts, the account is locked for 15 minutes. The lockout timestamp is stored in the database so it survives server restarts. The remaining lock time is included in the error message to help legitimate users understand what happened.

Where: `BACKEND/src/controllers/authController.js` — lines 130–

```
user.loginAttempts +=
if (user.loginAttempts >=
  user.lockUntil = Date.now() + 15 * 60 *
}
```

1.6 JWT Secret Enforcement

Before any JWT is signed, the code checks that `JWT_SECRET_KEY` is set in the environment and is at least 32 characters long. If it's missing or too short, the server throws an error immediately rather than using a weak or default secret.

Where: `BACKEND/src/controllers/authController.js` — lines 58–62

1.7 Two-Factor Authentication (TOTP)

Users can enable Time-Based One-Time Password (TOTP) 2FA using any standard authenticator app (Google Authenticator, Authy, etc.). The secret is generated using the `speakeasy` library and stored encrypted in the database with `select: false` .

Where: `BACKEND/src/controllers/authController.js` — lines 535–571

1.8 Secure 2FA Challenge Binding

When a user logs in with 2FA enabled, the server does not simply trust the `userId` submitted by the browser. Instead, it issues a short-lived, server-signed JWT "challenge token" (valid for 5 minutes) that binds the session to the correct user. The 2FA verification step then validates this token — if someone tries to swap in a different userId, the signature check will fail.

Where: `BACKEND/src/controllers/authController.js` — lines 145–

```
const MFA_CHALLENGE_PURPOSE = "login-
const MFA_CHALLENGE_TTL =
const signMfaChallenge = (user) =>
  { id: user._id.toString(), purpose:

  { expiresIn:
);
```

2. Authorization & Access Control

2.1 Session Verification Middleware (protect)

Every protected route passes through the `protect()` middleware before reaching its handler. This middleware calls the Ory Network API to verify the session cookie, and only continues if the session is valid and active. It then

attaches the verified user object to `req.user` so all subsequent handlers can trust it. Where: `BACKEND/src/middleware/authMiddleware.js` — lines 79–136

2.2 Role-Based Access Control (authorize)

The `authorize()` middleware sits after `protect()` and checks `req.user.role` against the list of allowed roles. If the user's role is not in the list, the request is rejected with a generic "Forbidden" message (no role name is disclosed to prevent information leakage).

Where: `BACKEND/src/middleware/authMiddleware.js` — lines 152–158

2.3 Server-Side Role Promotion

User roles are never trusted from the database alone. When a user logs in, the server cross-checks their email against the `ADMIN_EMAILS` environment variable. If it matches, the role is promoted to admin on the server side — no frontend or database value can grant admin access just by being set incorrectly.

Where: `BACKEND/src/middleware/authMiddleware.js` — lines 107–123

2.4 Object-Level Authorization (IDOR/BOLA Prevention)

Profile updates always use `req.user.id` (from the verified session) — never a user-supplied ID from the request body or URL. This means a user cannot modify another user's profile by guessing or forging an ID.

Where: `BACKEND/src/controllers/userController.js` — line 27

```
const user = await
User.findById(req.user.id);
```

2.5 Notification Authorization (BOLA)

When a user reads or hides a notification, the controller checks that the notification belongs to that user or is a broadcast. Users cannot interact with other users' private notifications.

Where: `BACKEND/src/controllers/notificationController.js` — lines 30–39

```
{ recipient: "all"
 },

{ recipient:     }
 req.user.id
```

2.6 Mass Assignment Protection

Admin user-update endpoints only accept an explicit whitelist of fields (`name` , `email` , `active` , `isEmailVerified` , `role`). The raw `req.body` object is never passed directly to `findByIdAndUpdate` , preventing an attacker from injecting privileged fields like `password` or `oryId` .

Where: `BACKEND/src/controllers/adminController.js` — lines 75–95

3. Rate Limiting & Brute-Force Protection

3.1 Strict Rate Limit on Authentication Endpoints

All authentication routes (`/login` , `/register` , `/forgot-password` , etc.) have a dedicated rate limiter capped at 20 requests per IP per 15 minutes. This slows down credential stuffing and brute-force attacks significantly.

Where: `BACKEND/src/routes/authRoutes.js` — lines 26–38

```
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000,
max: 20 });
```

3.2 General API Rate Limit

All other API routes are capped at 500 requests per IP per 15 minutes. This prevents general API abuse and DoS-style flooding while being high enough that normal dashboard usage (multiple concurrent API calls per page) doesn't hit the limit.

Where: `BACKEND/src/index.js` — lines 101–112

3.3 Progressive Request Slowdown

After 50 requests in 15 minutes, `express-slow-down` starts adding incremental delays (100ms × number of requests over the threshold) before responding. This discourages automated scanning and flooding without hard-blocking legitimate users.

Where: `BACKEND/src/index.js` — lines 116–123

3.4 Notification Polling Backoff

The frontend notification bell polls every 60 seconds (reduced from 30s). If the backend returns a 429 (rate limited), it backs off for 2 full minutes before retrying, so notification retries don't consume rate-limit budget needed for more critical calls like session verification.

Where: `frontend/components/NotificationBell.js` — lines 18–52

4. Input Validation & Injection Prevention

4.1 Global Input Sanitizer (NoSQL + XSS)

A custom middleware runs before every route handler and sanitizes `req.body` , `req.query` , and `req.params` . It:

- Deletes any key starting with `$` (blocks MongoDB operator injection like `{`
- `"password": { "$gt": "" } }`) Strips `<script>` blocks (XSS)

  Removes remaining HTML angle brackets `<` and `>` (XSS)

It intentionally skips secret fields like `password` and `csrf_token` so sanitization doesn't corrupt cryptographic values.

Where: `BACKEND/src/utils/sanitizer.js` — lines 29–116

4.2 Why express-mongo-sanitize Was Replaced

The popular `express-mongo-sanitize` package is incompatible with Express 5 because it tries to reassign `req.query`, which Express 5 made a read-only getter. Our custom sanitizer provides equivalent protection (plus XSS stripping) without this conflict, and is applied on every request.

Where: `BACKEND/src/index.js` — line 131 (comment explains the replacement)

4.3 Path Traversal / Zip Slip Prevention

When users upload a ZIP file of their source code, every file entry's path is normalized and checked before processing. Entries with paths that start with `../`, contain `/../`, or are absolute paths are silently skipped. This prevents a malicious ZIP from injecting file labels into the scanner that could manipulate AI output.

Where: `BACKEND/src/controllers/projectController.js` — lines 205–215

```
const normalizedEntryName =
path.posix.normalize(entry.entryName.replace(/\\/g, "/"));

if (normalizedEntryName.startsWith("../") || ...)
continue;
```

4.4 Proper HTML Sanitization for Newsletter (Parser-Based)

When admins send a newsletter, the HTML content goes through `sanitize-html` (a full DOM-parser-based sanitizer) rather than a simple regex. It enforces an explicit allowlist of safe tags and attributes, and blocks:

- All event handlers (`onerror`, `onload`, `onclick`, etc.)
- `javascript:` and `data:` URI schemes in links

Any tag or attribute not on the whitelist

Where: `BACKEND/src/controllers/adminController.js` — lines 40–78 (allowlist config) and lines 200–210 (usage)

5. Data Protection & Encryption

5.1 AES-256-CBC Encryption for Source Code

All uploaded source code is encrypted with AES-256-CBC before being written to the vault directory. Even if the server's file system is compromised, the source files are unreadable without the encryption key.

Where: `BACKEND/src/utils/vaultService.js` — lines 67–72

5.2 Unique IV Per Encryption

A new 16-byte cryptographically random IV is generated for every single encryption operation using `crypto.randomBytes(16)`. Using a static IV would make encrypted files susceptible to pattern analysis — two files with the same content would produce identical ciphertexts. Random IV prevents this entirely. Where: `BACKEND/src/utils/vaultService.js` — line 68

```
const iv =
crypto.randomBytes(16);
```

5.3 Memory-Hard Key Derivation (scrypt)

The encryption key is not the raw `VAULT_SECRET` string — it is derived from `VAULT_SECRET + VAULT_SALT` using `scryptSync`, which is a memory-hard key derivation function. This makes offline brute-force attacks against the secret much more expensive.

Where: `BACKEND/src/utils/vaultService.js` — line 49

```
const KEY = crypto.scryptSync(process.env.VAULT_SECRET,
SALT, 32);
```

5.4 Mandatory Vault Credentials

If `VAULT_SECRET` or `VAULT_SALT` are not set in the environment, the server throws an error at startup and refuses to run. There are no fallback defaults that could lead to weak or shared encryption in production.

Where: `BACKEND/src/utils/vaultService.js` — lines 44–45

5.5 Sensitive Fields Not Returned by Default

The `password`, `twoFactorSecret`, and `loginAttempts` / `lockUntil` fields all use `select: false` in the Mongoose schema, so they are excluded from all queries unless explicitly requested in code.

Where: `BACKEND/src/models/User.js` — lines 84, 133, 138

6. Security Headers

6.1 Helmet (Backend API)

The Express backend uses `helmet`, which sets over 17 HTTP security headers automatically. These include `X-Frame-Options` (clickjacking protection), `XContent-Type-Options` (MIME sniffing protection), `Strict-Transport-Security` (HTTPS enforcement), and a Content Security Policy for the API layer.

Where: `BACKEND/src/index.js` — lines 62–75

6.2 Frontend & Admin Security Headers

Both Next.js applications add HTTP response headers on every page:

- `X-Content-Type-Options: nosniff` — stops browsers from guessing file types
- `X-Frame-Options: DENY` — prevents the app from being embedded in iframes (clickjacking)
- `Referrer-Policy: strict-origin-when-cross-origin` — limits how much URL information is shared in referrer headers
  `Permissions-Policy` — disables camera, microphone, and geolocation APIs

Where: `frontend/next.config.mjs` — lines 13–18 | `admin/next.config.mjs` — lines 13–18

7. Session & Cookie Security

7.1 HttpOnly Cookies

Authentication sessions are managed by Ory Network, which issues session cookies with the `HttpOnly` flag set. This means client-side JavaScript cannot read the cookie — XSS attacks that steal `document.cookie` cannot obtain the session token.

Where: `BACKEND/src/index.js` — lines 28–30 (comment documents this)

7.2 CORS Restricted to Known Origins

The backend CORS policy only accepts requests from the frontend (`CLIENT_URL`) and admin (`ADMIN_URL`) origins, defined in environment variables. Credentials (cookies) are only allowed from these origins, so a malicious third-party site cannot make credentialed API requests.

Where: `BACKEND/src/index.js` — lines 91–103

7.3 CSRF Protection via Ory Flow Tokens

The Ory login flow issues a unique `csrf_token` for every login session. The frontend extracts this token from the flow's UI nodes and sends it with every form submission. Ory rejects any submission that doesn't include the correct token, preventing Cross-Site Request Forgery attacks.

Where: `frontend/app/login/page.js` — lines 72–77

8. API Security & Error Handling

8.1 Generic Error Messages (Anti-Enumeration)

Authentication error responses never reveal whether a login attempt failed because the email doesn't exist or because the password was wrong. Both cases return the same generic message. Registration also uses a generic message if an email is already taken. This prevents attackers from enumerating valid accounts. Where: `BACKEND/src/controllers/authController.js` — lines throughout the login and register functions

8.2 No Stack Traces in API Responses

All `catch` blocks log the real error server-side (for debugging) but return only a safe, generic message to the client. Internal error details, file paths, and stack traces are never exposed in API responses.

Where: All controller files — every `catch` block uses a helper like `sendAdminServerError` or a manual `res.status(500).json({ message: "..." })`

8.3 Ory Proxy Open Redirect Prevention

The Ory proxy route rewrites `Location` redirect headers from upstream Ory responses. It only forwards redirects that point back to known Ory URLs or known relative self-service paths. Any other `Location` value (which could be an attacker-injected redirect target) is silently dropped.

Where: `frontend/app/api/ory-api/[...paths]/route.js` — lines 112–123 |

`admin/app/api/ory-api/[...paths]/route.js` — same section 8.4 Proxy Timeout Protection

The Ory proxy uses an `AbortController` with a 30-second timeout on every upstream fetch. If the Ory Network doesn't respond within 30 seconds, the proxy returns a 504 error rather than hanging the user's browser indefinitely.

Where: `frontend/app/api/ory-api/[...paths]/route.js` — lines 83–100

9. Frontend-Side Security

9.1 Avatar URL Validation

Before rendering a user avatar `<img>` tag, the code checks that the URL uses the `https:` protocol. Any avatar URL using `http:` , `javascript:` , `data:` , or any other scheme is replaced with a safe placeholder. This prevents XSS via `javascript:` in image sources.

Where: `frontend/components/Header.js` — `safeAvatarUrl` function |
`admin/components/RootAdminLayout.js` — same pattern

9.2 Minimal Sensitive Data in localStorage

The Header component caches only non-sensitive user data in `localStorage` (`name` , `role` , `avatar`). Fields like `email` , `oryId` , and tokens are never stored in localStorage. If an XSS attack reads localStorage, it gets display-only information — nothing that can be used to hijack the session. Where: `frontend/components/Header.js` — the `localStorage.setItem("user", ...)` call

9.3 Admin HTML Preview Sanitization

When admins compose newsletters, the live preview pane in the browser sanitizes the HTML before rendering it with `dangerouslySetInnerHTML` . It strips scripts, event handlers, and iframes so a malicious payload in the preview cannot execute in the admin's own browser.

Where: `admin/app/newsletter/page.js` — `sanitizePreviewHtml` function

9.4 Download Filename Sanitization

When a user downloads a project report, the filename is sanitized by replacing non-alphanumeric characters with underscores. This prevents path traversal via crafted project names when the browser saves the file.

Where: `frontend/app/dashboard/projects/[id]/page.js` — filename sanitization on the download handler

9.5 ProtectedRoute Only Redirects on Real Auth Failures

The `ProtectedRoute` component only redirects to `/login` when the backend returns a genuine auth failure (HTTP 401 or 403). Transient errors like rate limiting
(429) or network timeouts don't trigger a redirect — the user stays on the page and the app retries when conditions improve. This prevents a rate-limit spike from accidentally locking authenticated users in an infinite login loop.

Where: `frontend/components/ProtectedRoute.js` — lines 35–45

10. Cloud & Environment Security

10.1 No Secrets in Source Code

All secrets (`VAULT_SECRET`, `VAULT_SALT`, `JWT_SECRET_KEY`, `ORY_SDK_URL`, `MONGODB_URL`, API keys) are stored in `.env` files that are excluded from version control via `.gitignore`. The application refuses to start if critical secrets are missing.

Where: `BACKEND/.env`, `frontend/.env`, `admin/.env` (not committed) | Validation in `BACKEND/src/utils/vaultService.js` line 44 and `authController.js` line 59

10.2 NEXT_PUBLIC Prefix for Client-Exposed Variables

Environment variables that need to be readable by the browser are explicitly prefixed with `NEXT_PUBLIC_`. Variables without this prefix (like `ORY_SDK_URL`, `MONGODB_URL`) are only available server-side and are never bundled into the browser JavaScript, keeping infrastructure details hidden from end users.

Where: `frontend/.env` — `NEXT_PUBLIC_ORY_SDK_URL`, `NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_ADMIN_URL`

10.3 ORY_SDK_URL Required in Production

The Ory proxy route checks for the `ORY_SDK_URL` environment variable at runtime. In production, if the variable is not set, the proxy returns a 500 configuration error immediately instead of silently falling back to a hardcoded URL (which could point to a different tenant). A development-only fallback exists to avoid blocking local development.

Where: `frontend/app/api/ory-api/[...paths]/route.js` — `resolveOrySdkUrl()` function

Summary Table

Practice	Category	File	Lines
bcrypt password hashing (cost 12)	Authentication	`BACKEND/src/models/User.js`	168–171
Timing-safe password compare	Authentication	`BACKEND/src/models/User.js`	186
Password field `select: false`	Data Protection	`BACKEND/src/models/User.js`	84
Account lockout (5 attempts, 15 min)	Brute-Force Prevention	`BACKEND/src/controllers/authController.js`	330–334
JWT secret validation at startup	Authentication	`BACKEND/src/controllers/authController.js`	58–62
TOTP 2FA (speakeasy)	MFA	`BACKEND/src/controllers/authController.js`	535–571

2FA challenge token binding	Authentication Bypass Prevention	`BACKEND/src/controllers/authController.js`	45–96
Session verification middleware	Authorization	`BACKEND/src/middleware/authMiddleware.js`	79–136
Role-Based Access Control	Authorization	`BACKEND/src/middleware/authMiddleware.js`	152–158
IDOR prevention (use session ID)	BOLA	`BACKEND/src/controllers/userController.js`	27
Notification BOLA check	BOLA	`BACKEND/src/controllers/notificationController.js`	30–39
Mass assignment protection	Authorization	`BACKEND/src/controllers/adminController.js`	75–95
Auth rate limit (20/15 min)	Rate Limiting	`BACKEND/src/routes/authRoutes.js`	26–38
General rate limit (500/15 min)	Rate Limiting	`BACKEND/src/index.js`	101–112
Progressive slowdown	Rate Limiting	`BACKEND/src/index.js`	116–123
NoSQL injection prevention ($key removal)	Injection	`BACKEND/src/utils/sanitizer.js`	95–101
XSS input sanitization	XSS	`BACKEND/src/utils/sanitizer.js`	66–70
Zip Slip / Path Traversal prevention	Path Traversal	`BACKEND/src/controllers/projectController.js`	205–215
Parser-based HTML sanitization (newsletter)	XSS	`BACKEND/src/controllers/adminController.js`	40–78, 200–210
AES-256-CBC source code encryption	Data Protection	`BACKEND/src/utils/vaultService.js`	67–72
Unique IV per encryption	Cryptography	`BACKEND/src/utils/vaultService.js`	68
scrypt key derivation	Cryptography	`BACKEND/src/utils/vaultService.js`	49
Vault credentials required at startup	Cloud Security	`BACKEND/src/utils/vaultService.js`	44–45
Helmet security headers (API)	Security Misconfiguration	`BACKEND/src/index.js`	62–75
Browser security headers (frontend/admin)	Security Misconfiguration	`frontend/next.config.mjs`	13–18
CORS restricted to known origins	API Security	`BACKEND/src/index.js`	91–103
CSRF via Ory flow tokens	CSRF	`frontend/app/login/page.js`	72–77
Generic error messages	Error Handling	All controllers	—
Ory proxy open redirect prevention	Open Redirect	`frontend/app/api/ory-api/[...paths]/route.js`	112–123
Proxy timeout (AbortController)	Resource Consumption	`frontend/app/api/ory-api/[...paths]/route.js`	83–100

Avatar URL validation (https only)	XSS	`frontend/components/Header.js`	safeAvatarUrl fn
Minimal localStorage data	XSS Impact Reduction	`frontend/components/Header.js`	localStorage call

Admin newsletter preview Practice sanitization	CategoryXSS	File`admin/app/newsletter/page.js`	sanitizePreviewHtml Lines fn
ProtectedRoute transient error handling	Availability	`frontend/components/ProtectedRoute.js`	35–45
No secrets in source code	Cloud Security	`.env` files	—
NEXT_PUBLIC prefix for browser vars	Cloud Security	`frontend/.env`	—